# Week 9 deployment packet

Risk scorer for APP-1001, Alice Chen, homeowners. The graded result is the rule score: 13, band low, factors `["base", "credit excellent"]`, `hard_refer []`.

# 1. IAM role and scoped policy (simulated)

Simulated. These roles were not created. This account cannot manage IAM. No `iam` API was called. There is no role ARN and no console screenshot. `APPROVED_REGION` and `ACCOUNT_ID` stay placeholders.

The task role is the identity of the running scorer. The execution role is a separate identity that would pull the image and write the log stream.

`create-role` and `put-role-policy` for `underwriting-risk-task-role` and `underwriting-risk-execution-role` were not run.

## Task role `underwriting-risk-task-role`

Trust principal `ecs-tasks.amazonaws.com`, action `sts:AssumeRole` only.

```
{
  "Versi
```

Scoped policy. One allow.

```
{
  "Versi
```

| Permission | Resource | Why it is necessary |
|---|---|---|
| bedrock:InvokeModel | arn:aws:bedrock:APPROVED_REGION::foundation-model/google.gemma-3-4b-it | The agent calls that one model and no other AWS API. |

| Permission | Why it is left off |
|---|---|
| bedrock:InvokeModelWithResponseStream | The process does not stream. It sends one prompt and reads one rationale. |
| bedrock:* | A wildcard would allow every Bedrock API and every model. The scorer needs one action on one model. |
| S3 | The process does not store objects. |
| IAM | The process does not change identities. |
| EC2 | The process does not manage the network. |
| ECR | The process does not pull images. Image pull belongs to the execution role. |
| CloudWatch Logs | The process does not create log streams. The awslogs driver belongs to the execution role. |
| Secrets Manager | The process does not read secrets. |

## Execution role `underwriting-risk-execution-role`

Same trust service, different role. Fargate would assume it to pull the private image and open the log stream.

```
{
  "Versi
```

| Permission | Resource | Why it is necessary |
|---|---|---|
| ecr:GetAuthorizationToken | * | Fargate would pull the private image. ECR requires Resource: "*" for this action. |
| ecr:BatchCheckLayerAvailability | repository underwriting-risk | Fargate would check that the layers of this one repository exist. |
| ecr:GetDownloadUrlForLayer | repository underwriting-risk | Fargate would obtain the layer URLs for this image. |
| ecr:BatchGetImage | repository underwriting-risk | Fargate would read the image manifest. |
| logs:CreateLogStream | log group /ecs/underwriting-risk | The task log driver would open one stream. |
| logs:PutLogEvents | log group /ecs/underwriting-risk | The task log driver would write the score line. |

| Permission | Why it is left off |
|---|---|
| bedrock:InvokeModel | The model call belongs to the task role. |
| bedrock:InvokeModelWithResponseStream | This role does not call the model, and the process does not stream. |
| bedrock:* | This role has no Bedrock work. |
| S3 | ECR hands Fargate pre-signed layer URLs. |
| logs:CreateLogGroup | The log group would be created before the task. |
| ecr:PutImage, ecr:CreateRepository, ecr:* | This role pulls one existing repository. |
| IAM, EC2, Secrets Manager | This role does not change identities, manage the network, or read secrets. |

# 2. Build, push, and run-task (not run)

Not run. This account has no working SSO session and cannot use ECR, ECS, or Fargate. No `ecr`, `ecs`, `logs`, or `sso` API was called. There is no push log and no run-task log. The 15 points for a live Fargate run are not claimed.

Bedrock model access for `google.gemma-3-4b-it` would have to be enabled in the approved region before the run. That enablement was not done.

## 1. Install the AWS CLI and log in with SSO

Not run.

```
aws configure sso --profile underwriting-risk
# start
```

## 2. Create the two roles

Not run. The task definition still names `underwriting-risk-task-role` and `underwriting-risk-execution-role`.

## 3. Create the ECR repository, build, tag, and push

Not run. A local image `underwriting-risk:phase1` exists with id `309f5a186dc2`. ECR credentials do not exist, so the repository was not created, the image was not tagged for ECR, and nothing was pushed. There is no push log.

```
aws ecr create-repository --repository-name underwriting-risk --profile underwriting-risk
docker b
```

## 4. Register the task definition

Not run. The file is `deploy/ecs/task-definition.json`. Fargate, `awsvpc`, 256 CPU, 512 memory, command `APP-1001`, `LLM_PROVIDER=bedrock`, log group `/ecs/underwriting-risk`.

```
{
  "famil
```

```
aws ecs register-task-definition --cli-input-json file://deploy/ecs/task-definition.json --profile underwriting-risk
```

## 5. Run the task

Not run.

```
aws ecs run-task \
  --clus
```

## 6. Read the score line

Not run. `aws logs tail /ecs/underwriting-risk` was not run. CloudWatch has no stream, and the block below is not a log line.

```
simulated result, task not executed
applicat
```

The score is the rule score. The model was not called, so this block has no rationale.

## 3. AWS task output for APP-1001 (not run)

Not run. The ECS task was not run. CloudWatch has no stream for this run. The block below is a simulated result, not a log line.

Input that would have been passed: APP-1001, Alice Chen, homeowners.

```
simulated result, task not executed
```

```
applicat
```

The score is the rule score. The model was not called, so this block has no rationale.

# 4. Subnet and security group

This is the network design. No VPC, subnet, security group, or endpoint was created. No EC2 API was called. The screenshot was not taken because this account has no VPC from this work to photograph.

The task is one shot. Nothing on the internet calls this scorer, so it runs in a private subnet. The design label is `subnet-PRIVATE`. That label is not a live subnet id. The route table has the VPC local route only. It has no route to the internet.

`assignPublicIp` is `DISABLED`. The task receives a private address and no public address.

```
{
  "awsvp
```

Security group `risk-agent-sg` (design label `sg-risk-agent`) has no inbound rules. Egress is TCP 443 only, toward the interface endpoints and the S3 gateway prefix list. There is no egress rule to the open internet.

| Direction | Protocol | Port | Destination | Why |
|---|---|---|---|---|
| Inbound | none | none | none | Nothing connects to a one-shot scorer. |
| Egress | TCP | 443 | Bedrock runtime interface endpoint | The task role calls bedrock:InvokeModel. |
| Egress | TCP | 443 | ecr.api interface endpoint | The execution role authenticates and reads the image API. |
| Egress | TCP | 443 | ecr.dkr interface endpoint | Fargate downloads the image layers. |
| Egress | TCP | 443 | CloudWatch Logs interface endpoint | The awslogs driver writes the score line. |
| Egress | TCP | 443 | S3 gateway prefix list | ECR layer blobs are fetched through the S3 gateway. |

`risk-endpoint-sg` allows inbound TCP 443 from `risk-agent-sg` only.

| Endpoint | Type | Service name |
|---|---|---|
| Bedrock runtime | interface | com.amazonaws.APPROVED_REGION.bedrock-runtime |
| ECR API | interface | com.amazonaws.APPROVED_REGION.ecr.api |
| ECR Docker | interface | com.amazonaws.APPROVED_REGION.ecr.dkr |
| CloudWatch Logs | interface | com.amazonaws.APPROVED_REGION.logs |
| S3 | gateway | com.amazonaws.APPROVED_REGION.s3 |

A NAT gateway would also let the task run. That path opens the whole internet. The endpoint design is the one this task uses, because the task only needs Bedrock, ECR, CloudWatch Logs, and S3.

The `create-vpc`, `create-subnet`, `create-security-group`, and `create-vpc-endpoint` commands were not run. There is no VPC id and no screenshot.

## 5. Ollama setup and the local call

`ollama list` on this machine. The model was already local, so it was not pulled again. The listener is loopback only.

```
NAME         ID              SIZE      MODIFIED
gemma3:4
```

The agent posts one chat request to `http://127.0.0.1:11434/api/chat` with model `gemma3:4b`. `LLM_PROVIDER=ollama`. The function in `src/underwriting/llm.py`:

```
OLLAMA_URL = "http://127.0.0.1:11434/api/chat"
OLLAMA_M
```

The shared prompt asks for a short JSON rationale only. The rule score is already decided.

# 6. Ollama output for APP-1001

Same input as the AWS task: APP-1001, Alice Chen, homeowners. This block is the unedited stdout of `LLM_PROVIDER=ollama python -m underwriting APP-1001`.

```
application: APP-1001
                                                
```

# 7. Architecture

A grader can score this section without reading the code. The application is one homeowners risk score for APP-1001, Alice Chen. The rule score is 13, band low. The model writes a short rationale only. Two places can run that same program. They do not draw the same boundary around the application text.

## AWS path

This path was not executed. No ECS task was started, and no Bedrock call was made.

The program would run as one Fargate container in the private subnet `subnet-PRIVATE`. `assignPublicIp` is disabled. Nothing on the internet connects to the task. Security group `risk-agent-sg` has no inbound rules. Its egress is TCP 443 to the Bedrock runtime, ECR, CloudWatch Logs, and the S3 gateway.

The identity of the running program is the task role `underwriting-risk-task-role`. That role can call `bedrock:InvokeModel` on `google.gemma-3-4b-it` and nothing else. A second role, `underwriting-risk-execution-role`, pulls the image and writes the log stream. The program does not use the execution role to call the model.

The image comes from the private ECR repository `underwriting-risk`. The model call leaves the container. The prompt, which contains the application id, the name, the product, and the scored facts, is sent to the Bedrock runtime endpoint. The application text would be inside AWS, and it would not stay inside the container only. Bedrock holds the prompt for the duration of the invoke. The awslogs driver sends the score line to CloudWatch Logs in the group `/ecs/underwriting-risk`.

The container has no public address. The private subnet still reaches Bedrock, ECR, and CloudWatch through interface endpoints, and S3 through a gateway endpoint. Those endpoints are inside the VPC, and the application text still leaves the container for a managed model endpoint.

## On-prem path

This path was executed on this machine.

One process runs `python -m underwriting APP-1001`. The model weights for `gemma3:4b` are on local disk. Ollama listens on `127.0.0.1:11434`, and the process posts the prompt to that loopback address. The prompt does not cross a network interface that another host can route to.

This path has no IAM role, no subnet, no security group, no ECR, no Bedrock, and no CloudWatch. The score line is the process stdout. The recorded result is score 13, band low, factors `["base", "credit excellent"]`, `hard_refer []`.

## What an air gap removes

An air gap removes `docker pull`, `pip install`, `ollama pull`, ECR, Bedrock, CloudWatch, and SSO. After the network is cut, none of those calls can succeed. The image and the Gemma weights have to be loaded onto the host before the cut. The running process then uses only the local files and the loopback listener.

A client asks for this when the application is insurance PII and the contract or the regulator requires the data to stay on their network. APP-1001 carries a person's name and the underwriting facts that produced the score. On the AWS path that text leaves the container for Bedrock. On the air-gapped path it stays on the host.

# 8. Comparison and recommendation

The recommendation names the client and the deployment. Cost, complexity, and output quality are the reasons. No dollar figure is given for a Bedrock call. That call was not made.

The cost shape on AWS is one `RunTask` that starts, prints the score, and exits. It is not a service left on. Fargate would bill the seconds the task runs. Bedrock would add a per-token charge for the one rationale. CloudWatch would keep the score line as the audit trail. On the laptop, Ollama has no per-token bill. The cost is the host and the disk for the `gemma3:4b` weights.

The AWS path has more moving parts: two IAM roles, a private subnet, interface endpoints, an ECR repository, a task definition, and a log group. The laptop path is one process and a loopback listener. The Fargate deployment was not run.

Output quality is scored from the real Ollama response only. For APP-1001 the model wrote that Alice Chen's excellent credit and the base factor support a low band at score 13. Those are the rule factors, `[ "base", "credit excellent"]`. The Bedrock rationale was not produced, so the two wordings are not compared. Both sides use Gemma 3 4B and the same prompt. The graded correct result on either side is the rule score, 13, band low.

## Strict data residency

Use Ollama, or the same open weights on the client's own hosts.

A private subnet plus Bedrock still sends the application to a managed endpoint. The prompt leaves the container for `bedrock-runtime`. That fails a residency requirement even when the IAM role is perfect. APP-1001 is insurance PII: a person's name and the facts behind the score. The contract or the regulator that requires the data to stay on the client network is the reason to keep the weights and the process inside that network.

## No residency requirement

Use Fargate plus Bedrock once `underwriting-risk-task-role` exists.

This client can accept the application text inside AWS. The extra moving parts buy a task role scoped to one model, a private subnet with no public address, and a CloudWatch log of the score line. The charge is one exiting task plus the tokens for one rationale. The rule score stays 13, band low. This deployment was not run.